Fine-tuning a Lean-4-only GPT: does RLHF beat supervised fine-tuning?

nanoGPT (0.8M parameters) · pretraining, SFT, RLHF and alternatives, measured with a proof checker and Lean 4

Status (4 Nov 2026). The first run (section 4.1) and the replication's training and test evaluation (section 4.2) are complete: 14 checkpoints, 2,000 test problems each, every answer compiled by Lean 4. The detailed analysis is being written and will replace this version at the same link.

Summary

The model. A 0.8M-parameter GPT (the nanoGPT architecture) that I pretrained, fine-tuned and post-trained from scratch on nothing but Lean 4: propositional and first-order proofs, `Nat` arithmetic, and linear equations over `Int`. There is no natural language and no safety training anywhere in the pipeline.

What I changed: the post-training method. From one supervised fine-tuned (SFT) checkpoint I compared RLHF, which trains a Bradley-Terry reward model on checker-labelled comparisons of the model's own answers and then runs PPO against it with a KL penalty, with PPO run directly against the proof checker. The replication adds rejection-sampling fine-tuning (keep the sampled proofs that check, fine-tune on them) and a "more SFT" control. All arms that sample get the same sample budget.

How I measured it. Greedy accuracy on held-out problems. The train/test split hashes a canonical form of each problem, so renamed or respelled copies of test problems never appear in training. Answers are judged by checkers that are fuzzed against real Lean 4. In the replication, every answer is also compiled by Lean, results are compared with paired bootstrap confidence intervals, and each arm is run with three seeds. I also tracked the reward model's accuracy on held-out comparisons, and the reward against true accuracy during PPO.

What I learned.

- **No post-training method beat SFT.** In the replication (2,000 test problems, Lean-verified, 3 seeds, paired bootstrap 95% CIs), against SFT at 34.7%:
  - RLHF lost 1.9 points [−3.2, −0.7], p = 0.003;
  - PPO against the checker changed nothing (−0.4 [−1.5, +0.6]);
  - rejection-sampling fine-tuning lost 2.5 points [−3.8, −1.2];
  - training SFT for 600 more steps gained 0.7 [−0.4, +1.9]. In the first run, RLHF got 38.7% against SFT's 38.0%, inside the ±1.4-point standard error.
- **The reward model was the bottleneck.** It ranked only 57% (first run) and 55.8% (replication) of held-out pairs correctly while fitting its training pairs. PPO against it lowered true accuracy. With a looser KL penalty in the first run, the reward rose while checked accuracy fell from 41% to 8%.
- **Small validation sets mislead.** On the 300-problem validation split, 11 of the 12 post-trained runs scored above SFT (by 0.7 to 5.0 points) and one tied. On the 2,000-problem test set, all nine RL runs scored below it. In the first run, an iterate selected for being 4.6 points better on validation did not hold up on test.
- **The checker held up.** Across 28,000 evaluated answers, it never accepted an answer Lean rejected.

1. The model and the data

A problem is a Lean 4 declaration cut in two: the model sees the header and writes the rest, and the whole declaration must compile.

domain	the model sees	and writes
prop	`example (p q s : Prop) : p ∧ (¬p ∧ ¬q) ∧ s → ¬p :=`	`fun h0 => h0.2.1.1`
fol	`example (α : Type) (P R : α → Prop) (q : Prop) : (∃ x, P x) → (∀ x, P x → q ∧ R x) → ∃ x, q ∧ R x :=`	`fun ⟨x, h0⟩ h1 => ⟨x, h1 x h0⟩`
nat	`example : (10 - 10 * 6) % 15 + 7 =`	`7 := by decide`
linear	`example (x : Int) (h : 7 * x - 14 = x + 10) : x =`	`4 := by omega`

- **Model.** A character-level GPT-2-style transformer with 4 layers, 4 heads, width 128, a 256-token context, tied embeddings and 803,072 parameters.
- **Tokens.** Sequences are `<bos> header <ans> answer <eos>`. Pretraining documents are whole declarations.
- **Checkers.** Each domain has a generator and a checker: a Python implementation of the fragment of Lean 4 the domain uses, fast enough to judge every sample during training. A checker must be sound with respect to Lean: it may reject a proof Lean accepts, never the reverse.
- **Testing the checkers.** The tests compile generated declarations, and hundreds of thousands of mutated answers the checkers accept, with Lean 4.34. A JavaScript port of every checker drives an in-browser chat page, and the tests check that it agrees with the Python one.

2. The training pipeline

stage	data	objective
pretraining	160k whole declarations (train split)	next-token cross-entropy
SFT	80k (header, shortest reference proof) pairs	cross-entropy on the answer only
reward model	6,000 prompts × 8 SFT samples, pairs ranked by the checker (a correct proof beats an incorrect one; between two correct proofs, the shorter wins)	Bradley-Terry, −log σ(r_chosen − r_rejected)
RLHF	train-split headers	clipped PPO on RM score − β·KL(π ‖ π_SFT), GAE, critic initialised from the RM
PPO vs checker	the same	clipped PPO on ±1 from the checker − β·KL
RFT (replication)	the policy's own samples that check	cross-entropy, a few epochs per round
more SFT (replication, control)	fresh reference proofs	cross-entropy

The checker labels the comparisons in place of human raters. This makes them noise-free, so any gap between the reward model and the checker is the reward model's own error. PPO optimises the learned reward model, not the checker, exactly as RLHF with human labels does.

3. How it is measured

- **Held-out problems.** Problems are split train/val/test (90/5/5) by a hash of their canonical form. For logic, atoms are renamed in order of appearance and bound variables normalised. For arithmetic, commuted

operands and swapped sides of an equation are identified. A respelled test problem therefore cannot leak into training.

- **Metric.** Greedy pass@1. An answer counts only if it is finished and accepted.
  - First run: judged by the checkers.
  - Replication: also compiled with Lean 4. Checker false positives and false negatives are counted, and every model is scored on the same 2,000 test problems (500 per domain).
- **Statistics** (replication). Wilson intervals for each accuracy. Paired bootstrap intervals and exact McNemar tests for each difference from SFT. Three training seeds per arm, plus sampled pass@k (16 samples) to separate gains in reliability from gains in what the model can produce at all.
- **Diagnostics.**
  - The reward model's pairwise accuracy and AUC on prompts it was not trained on.
  - The reward model's score against checked accuracy across PPO iterations, to detect reward hacking.

4. Results

4.1 First run (complete)

Accuracy on 1,200 held-out test problems (300 per domain), greedy decoding, judged by the checkers:

checkpoint	prop	fol	nat	linear	all
pretrained (completing the declaration as a document)	29.3	54.0	16.0	8.0	26.8
SFT	47.0	67.7	21.7	15.7	38.0
RLHF (PPO against the reward model; iterate chosen on val, 90 of 150)	46.7	67.0	21.0	20.0	**38.7**
RLHF, last iterate	44.7	65.3	20.0	17.0	36.7
PPO against the checker itself (iterate 100)	47.0	67.3	24.0	14.3	38.2

- **Pretraining and SFT do the work.** The base model already proves a quarter of the problems by completing them as documents; SFT adds 11 points.
- **RLHF does not measurably improve on SFT.** The +0.7 points is within noise (the standard error on 1,200 problems is about 1.4 points), and the last iterate is worse.
- **The reward model is the bottleneck.** It was trained on 22,009 checker-labelled comparisons and ranks held-out pairs correctly 57% of the time. Judging whether `(7 + 5) * 3 - 4 = 34` is right is as hard for it as for the policy.
- **Reward hacking with a looser leash.** An earlier configuration, with a weaker reward model and a looser KL penalty, saw the reward model's score rise while checked accuracy fell from 41% to 8%. The run above uses the settings that stayed stable: learning rate 5e-6, KL coefficient 0.1, 2 PPO epochs.
- **Selection on validation did not transfer.** The iterate kept looked 4.6 points better than SFT on the val split, but it was the best of 16 noisy evaluations and the gain was not there on test.
- **PPO itself works.** On a synthetic reward (short answers) it goes from 30% to 97% in 30 iterations, and a single update raises the log-probability of correct proofs relative to wrong ones. Optimising the checker

directly was no better than the learned reward: at 0.8M parameters, RL mostly redistributes probability among answers the SFT model already produces.

4.2 Replication (complete)

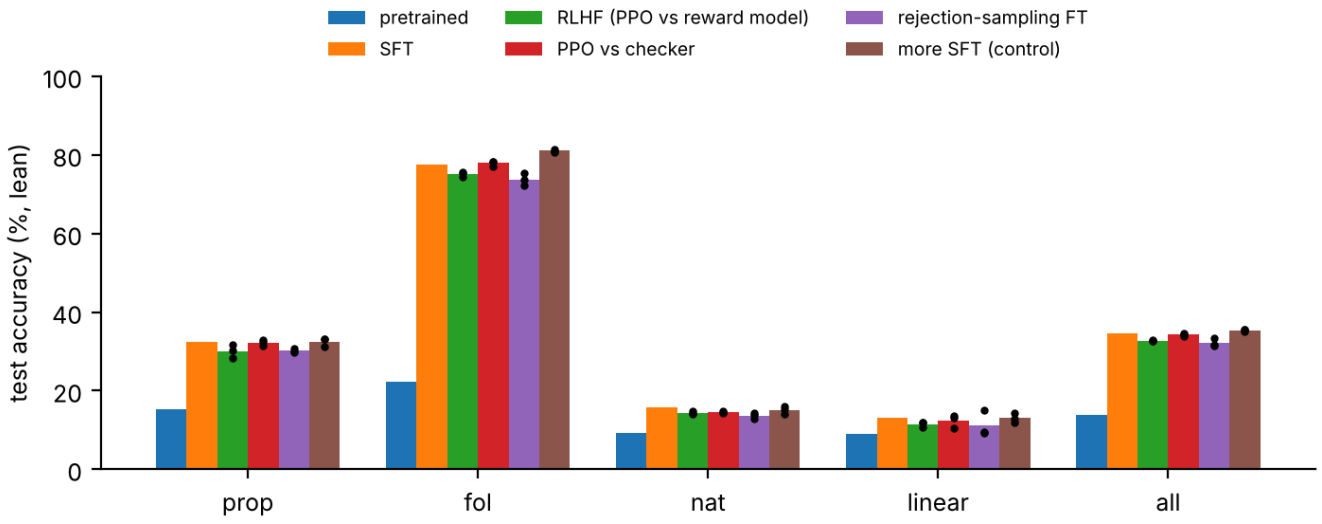
The replication fixes the first run's weak points:

- three seeds per arm, and the last iterate always (no selection on val);
- 2,000 test problems instead of 1,200;
- every answer compiled with Lean 4;
- paired statistics;
- two more arms: rejection-sampling fine-tuning, and "more SFT" on fresh references.

The problem generators were rebuilt for this run, so absolute accuracies are not comparable with section 4.1.

Test accuracy, Lean-verified, on 500 problems per domain. Arms show the mean ± standard deviation over 3 seeds; the difference from SFT pools the seeds, with a paired bootstrap 95% CI and a paired p-value:

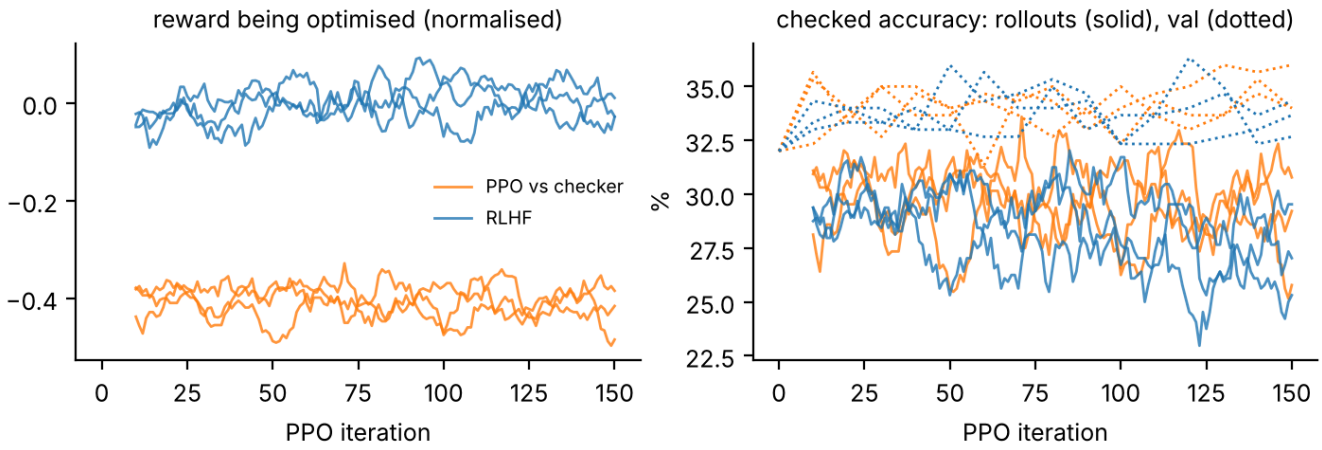
checkpoint	prop	fol	nat	linear	all	Δ all vs SFT [95% CI]	p
pretrained	15.2	22.4	9.2	9.0	14.0		
SFT	32.4	77.6	15.8	13.0	34.7		
RLHF (PPO vs reward model)	30.1 ± 1.8	75.1 ± 0.6	14.4 ± 0.4	11.5 ± 0.8	32.8 ± 0.2	−1.9 [−3.2, −0.7]	0.003
PPO vs checker	32.3 ± 0.8	77.9 ± 0.8	14.5 ± 0.3	12.4 ± 1.7	34.3 ± 0.4	−0.4 [−1.5, +0.6]	0.43
rejection-sampling FT	30.2 ± 0.5	73.8 ± 1.6	13.6 ± 0.7	11.3 ± 3.2	32.2 ± 1.1	−2.5 [−3.8, −1.2]	0.0002
more SFT (control)	32.5 ± 1.2	81.1 ± 0.3	15.1 ± 1.0	13.0 ± 1.1	35.4 ± 0.2	+0.7 [−0.4, +1.9]	0.21



Test accuracy by domain; dots are individual seeds.

- **Reward model.** It was trained on 25,622 comparisons, with 1,161 held out from 251 prompts. It ranked 55.8% of the held-out pairs correctly, and its score separated correct from incorrect answers with an AUC of 0.62. By the end of training, its training loss was 0.32 against a held-out loss of 1.02.
- **Validation versus test.** SFT scored 32.0% on the 300-problem val split and 34.7% on test. Every arm's final iterate scored 32.0–37.0% on val, all at or above SFT, yet every RL arm was below SFT on test.

- **Lean audit.** Across all 28,000 evaluated answers (8,174 distinct declarations), the checker never accepted an answer Lean rejected. Lean accepted 4 answers the checker had rejected.



Left: the reward PPO optimises. Right: checked accuracy of the rollouts (solid) and on val (dotted).

5. What I learned

1. **RLHF is only as good as its reward model, and a reward model can be confidently wrong.** With noise-free labels and 26k comparisons, a reward model the size of the policy still did not learn to judge proofs: it fit its training pairs and stayed near chance on new prompts. PPO then had little to optimise, and when the KL leash was loosened it optimised the errors instead.
2. **Measure the measurement.** Held-out pairwise accuracy of the reward model, and reward against true accuracy during PPO, explained the result before the test numbers did. A training curve that only shows the reward would have shown the hacking run "improving".
3. **Small gains need paired statistics and seeds.** A 4.6-point validation gain was selection noise, and a 0.7-point test gain was inside the standard error. The replication therefore fixes the test set, uses paired tests and three seeds, and drops selection on val.
4. **Contamination hides in respellings.** Hashing raw prompts let about a fifth of arithmetic and a seventh of linear-equation test problems have a commuted or side-swapped twin in training. Splitting on a canonical form closes that.
5. **Check the checker.** Fuzzing the checkers against real Lean found inputs they accepted and Lean rejected: non-space whitespace, and headers deep enough to exhaust Lean's recursion limit. An RL policy optimising a checker could find such holes, so the replication compiles every evaluated answer with Lean.

Reproducing

Code, tests and the experiment script: https://github.com/rbahl1/nanogpt. `./run_experiment.sh` runs the whole replication on a 4-core CPU in a few hours, and every long stage resumes exactly after an interruption.